1. What is U-Net? Implement U-Net for a Simple Application

Definition

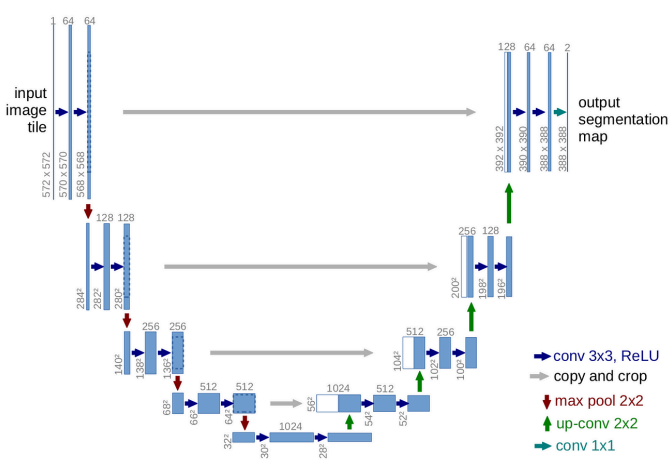
U-Net is a Convolutional Neural Network architecture designed primarily for image segmentation tasks. It was introduced in the paper:

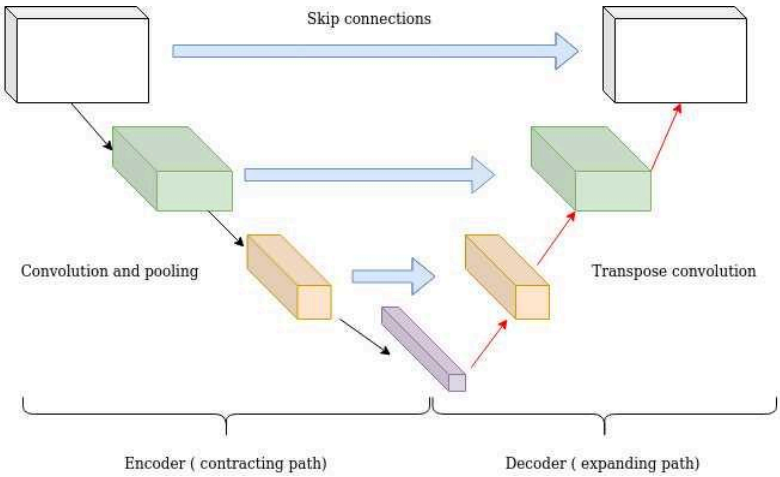
U-Net: Convolutional Networks for Biomedical Image Segmentation

The architecture is called U-Net because of its U-shaped structure. It consists of:

1. Contracting Path (Encoder) – extracts features and reduces spatial dimensions.
2. Bottleneck – connects encoder and decoder.
3. Expanding Path (Decoder) – restores spatial resolution.
4. Skip Connections – connect encoder layers to corresponding decoder layers to preserve spatial information.

U-Net Architecture Diagram





Application: Road Segmentation from Satellite Images

Objective:
To segment roads from satellite imagery using U-Net.

PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()

        def conv_block(in_c, out_c):
            return nn.Sequential(
                nn.Conv2d(in_c, out_c, 3, padding=1),
                nn.ReLU(),
                nn.Conv2d(out_c, out_c, 3, padding=1),
                nn.ReLU()
            )

        self.enc1 = conv_block(3, 64)
        self.enc2 = conv_block(64, 128)

        self.pool = nn.MaxPool2d(2)
```

```
        self.bottleneck = conv_block(128, 256)

        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec2 = conv_block(256, 128)

        self.up1 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec1 = conv_block(128, 64)

        self.out = nn.Conv2d(64, 1, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))

        b = self.bottleneck(self.pool(e2))

        d2 = self.up2(b)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, e1], dim=1)
        d1 = self.dec1(d1)

        return torch.sigmoid(self.out(d1))
```

---

2. Implementation of Hierarchical RAG

Concept

Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. The model retrieves relevant documents and then generates answers based on retrieved context.

Hierarchical RAG works in multiple levels:

- Level 1: Retrieve document category
- Level 2: Retrieve relevant section
- Level 3: Generate response

Example: University Information System

```
documents = {
   "Admissions": {
      "Eligibility": "Minimum 60 percent in 12th grade.",
      "Fees": "Annual tuition fee is 2 lakh INR."
   },
   "Hostel": {
      "Rooms": "Single and shared rooms available.",
      "Rules": "Entry allowed before 10 PM."
   }
}
```

Hierarchical retrieval function:

```
def hierarchical_retrieval(query):
   for category in documents:
      if category.lower() in query.lower():
         for section in documents[category]:
            if section.lower() in query.lower():
               return documents[category][section]
   return "Information not found."
```

Response generation:

```
query = "What is the fees in admissions?"
context = hierarchical_retrieval(query)
response = "Based on university records: " + context
print(response)
```

---

3. Generation of Synthetic Dataset

Project Title: Smart Traffic Prediction System

Steps:

1. Identify required features (hour, day, weather, vehicle count).
2. Generate random but realistic values.
3. Store the dataset in CSV format.

Implementation:

```
import pandas as pd
import numpy as np

np.random.seed(42)

data = {
    "hour": np.random.randint(0, 24, 1000),
    "day": np.random.randint(1, 8, 1000),
    "weather": np.random.choice(["Sunny", "Rainy", "Cloudy"], 1000),
    "vehicle_count": np.random.randint(50, 500, 1000)
}

df = pd.DataFrame(data)
df.to_csv("traffic_data.csv", index=False)

print(df.head())
```

---

4. Entropy Loss for AE and VAE (Using KL Divergence)

Autoencoder (AE)

Loss function is reconstruction loss, commonly Binary Cross Entropy (BCE):

$$
L = - \sum x \log(\hat{x})
$$

Variational Autoencoder (VAE)

Total loss consists of:

1. Reconstruction Loss
2. KL Divergence

KL Divergence formula:

$$
KL(q(z|x) || p(z)) = -\frac{1}{2} \sum (1 + \log\sigma^2 - \mu^2 - \sigma^2)
$$

Implementation:

```
def vae_loss(recon_x, x, mu, logvar):
    BCE = F.binary_cross_entropy(recon_x, x, reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KLD
```

---

5. Step-by-Step Stable Diffusion Process

Stable Diffusion is based on Latent Diffusion Models introduced in:

High-Resolution Image Synthesis with Latent Diffusion Models

Example: Generate "A futuristic smart city at night"

Step 1: Text Encoding

The text prompt is converted into embeddings using a text encoder.

Step 2: Forward Diffusion

Noise is gradually added to an image over multiple time steps.

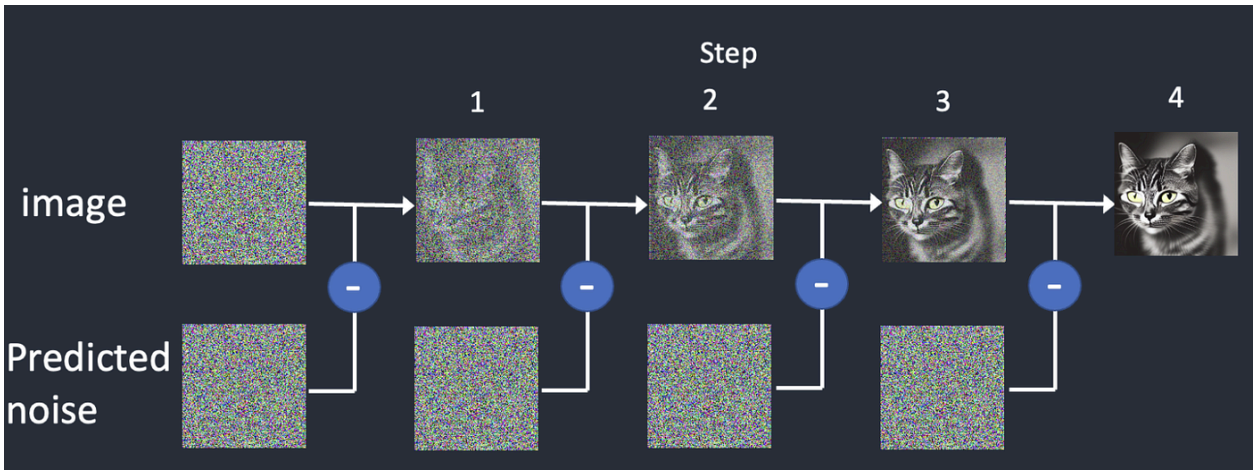
Step 3: Reverse Diffusion

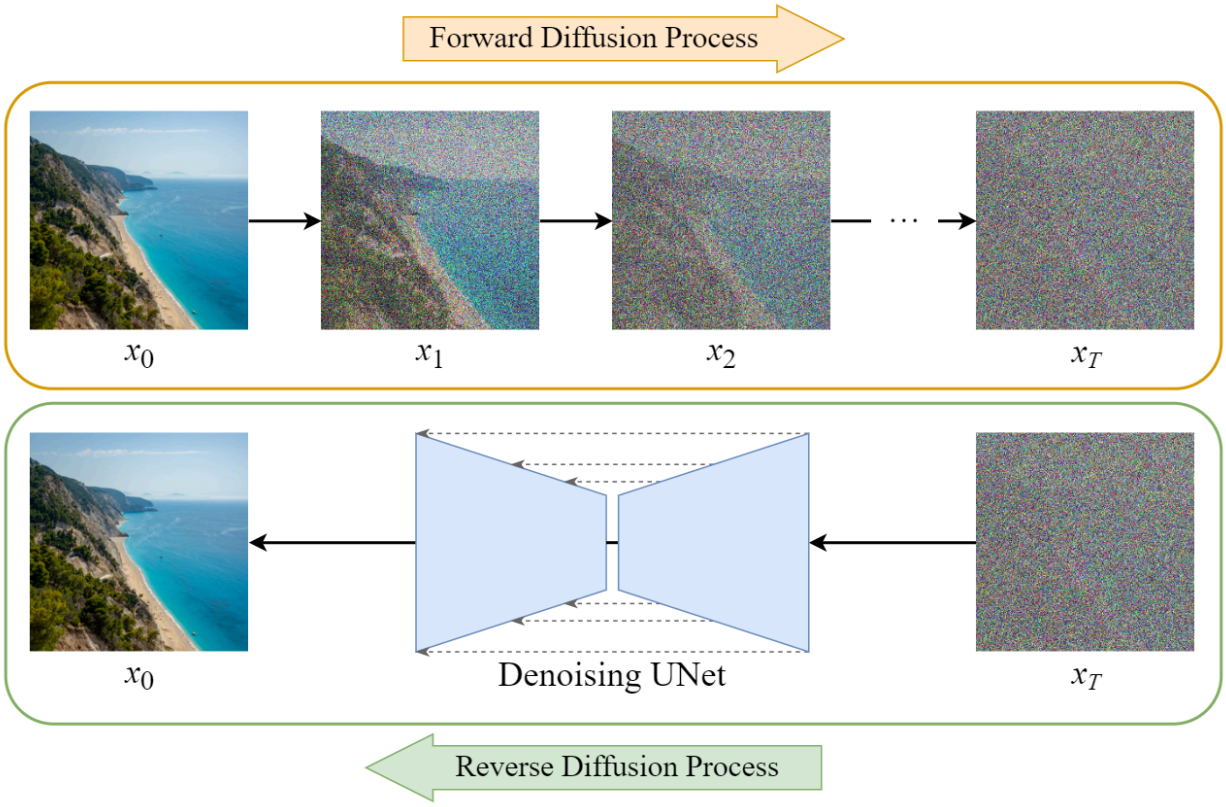
The model predicts and removes noise iteratively.

Step 4: Decoding

The final latent representation is decoded into an image.

Stable Diffusion Process Diagram





Simplified Diffusion Code Example

```
import torch

x = torch.randn(1, 3, 64, 64)

for t in range(10):
    noise = torch.randn_like(x)
    x = x - 0.1 * noise

print("Output shape:", x.shape)
```

Denoising Techniques

1. Gaussian noise reduction
2. Reverse diffusion sampling
3. Denoising autoencoders
4. Score-based noise prediction